

Recursive Queries in SQL

Expressive Power of SQL and Relational Algebra

DSC 100 – Introduction to Data Management

Contents

1	Expressive Power of SQL	3
1.1	Computable Functions and Turing Machines	3
1.2	Connectivity Queries in SQL	3
2	Transitive Closure of a Graph	4
2.1	Definitions	4
2.2	Iterative Construction	5
3	Recursive Queries in SQL	5
3.1	Limitation of Standard SQL	5
3.2	Recursive Common Table Expressions (CTEs)	5
3.3	Worked Examples	5
4	Pitfalls and Problematic Examples	7
5	Query Evaluation	7
5.1	Relational Algebra and Physical Operators	7
5.2	Hash Table Review	8
5.3	Combining Operators	8
5.4	Physical Plan Decisions	8
6	Data Storage and Indexing	9
6.1	Memory Hierarchy	9
6.2	Disk Organization	9
6.3	Data Files	9
6.4	Indexes	9
6.5	Index Selection Guidelines	10

7	Design Theory	11
7.1	Database Design Process	11
7.2	Anomalies in Unnormalized Schemas	11
7.3	Relation Decomposition	11
7.4	Functional Dependencies (FDs)	12
7.5	Inference of FDs: Closure	12
7.6	Keys and Superkeys	12
7.7	Eliminating Anomalies via Normalization	13
8	Entity–Relationship (E/R) Diagrams	13
8.1	Motivation	13
8.2	Basic Concepts	13
8.3	Keys in E/R Diagrams	13
8.4	Mathematical View of Relationships	14
8.5	Cardinality Constraints	14
8.6	Attributes on Relationships	14
8.7	Multi-way (Higher-arity) Relationships	14
8.8	Converting Multi-way to Binary Relationships	14
8.9	Design Principles	15
8.10	Mapping E/R Diagrams to Relational Schemas	15
8.11	Modeling Subclasses and Union Types	15
8.12	Weak Entity Sets	15
9	Boyce–Codd Normal Form (BCNF)	16
9.1	Summary	16
9.2	Definition of BCNF	16
9.3	BCNF Decomposition Algorithm	16
9.4	Example: A “Bad” FD	17
9.5	BCNF Decomposition: Person–Phone–City	17
9.6	Example: Two-Step BCNF Decomposition	17
9.7	Example: $R(A, B, C, D)$ with $A \rightarrow B$, $B \rightarrow C$	18
9.8	Lossless vs. Lossy Decomposition	18
9.9	Guaranteed Lossless Decomposition	18
9.10	Testing Lossless Join: The Chase Test	19
9.11	Schema Refinements and Normal Forms	19

1 Expressive Power of SQL

1.1 Computable Functions and Turing Machines

Definition 1.1 (Computable Function). A function $f : \mathbb{N} \rightarrow \mathbb{N}$ is *computable* if there exists a systematic, mechanical procedure (an algorithm) that, given any input x , eventually halts and returns the correct output $f(x)$.

Definition 1.2 (Turing Machine). A *Turing machine* is an abstract computational model consisting of

- an infinite tape divided into cells,
- a read/write head that can move left or right,
- a finite set of states, and
- a transition function driving moves and state changes.

It can simulate any algorithm that halts on every input.

SQL and Turing-completeness Full programming languages (C, Java, etc.) can express all computable functions, but *standard* SQL cannot, because it lacks recursion and thus cannot simulate arbitrary Turing-machine computations. 22

1.2 Connectivity Queries in SQL

We consider the classic “flight connectivity” problem on a table

`flight(from, to)`

asking: *Is there a way to get from City1 to City2?*

Example 1.1 (Direct Flight). `SELECT *
FROM flight f
WHERE f.from = 'City1'
AND f.to = 'City2';`

This answers reachability by a single direct flight.

Example 1.2 (At Most One Stopover). `SELECT x.from, y.to`
`FROM flight x, flight y`
`WHERE x.from = 'City1'`
`AND x.to = y.from`
`AND y.to = 'City2';`

This finds a route via exactly one intermediate city.

Example 1.3 (At Most Two Stopovers). `-- Direct:`

`SELECT * FROM flight f`
`WHERE f.from = 'City1'`
`AND f.to = 'City2';`

`-- One stop:`

`SELECT x.from, y.to`
`FROM flight x, flight y`
`WHERE x.from = 'City1'`
`AND x.to = y.from`
`AND y.to = 'City2';`

`-- Two stops:`

`SELECT x.from, z.to`
`FROM flight x, flight y, flight z`
`WHERE x.from = 'City1'`
`AND x.to = y.from`
`AND y.to = z.from`
`AND z.to = 'City2';`

Requires $k + 1$ copies of the table to allow up to k stops—unscalable for arbitrary k . 55

2 Transitive Closure of a Graph

2.1 Definitions

Definition 2.1 (Binary Relation). A *binary relation* R on a set X is a subset of $X \times X$.

Definition 2.2 (Transitive Relation). A relation $R \subseteq X \times X$ is *transitive* if

$$(a, b) \in R \wedge (b, c) \in R \implies (a, c) \in R.$$

Definition 2.3 (Transitive Closure). The *transitive closure* of R is the smallest transitive relation R^+ containing R . Equivalently, it contains all pairs of nodes $\langle a, b \rangle$ connected by some directed path. 66

2.2 Iterative Construction

Let

$$T_1 = R, \quad T_k = T_{k-1} \cup \{(x.a, y.b) \mid (x.a, x.b) \in R, (y.a, y.b) \in T_{k-1}, x.b = y.a\}.$$

Then $R^+ = \bigcup_{k=1}^{|X|} T_k$. We stop once no new pairs appear.

3 Recursive Queries in SQL

3.1 Limitation of Standard SQL

Basic SQL has no mechanism to iterate until fixpoint, so it cannot express transitive closure or similar recursive queries.

3.2 Recursive Common Table Expressions (CTEs)

Modern SQL adds `WITH RECURSIVE` to define a fixpoint:

```
WITH RECURSIVE t AS (  
  -- Base case:  
  SELECT * FROM G  
  UNION  
  -- Recursive case:  
  SELECT x.a, y.b  
    FROM G x, t y  
   WHERE x.b = y.a  
)  
SELECT * FROM t;
```

Base case: initializes t with direct edges. **Recursive case:** joins new with previous to add indirect paths. Termination occurs when no new rows can be added.

3.3 Worked Examples

Example 3.1 (Prerequisites). Given `prereq(course_id, prereq_id)`, compute all (direct and indirect) prerequisites of ‘DSC-100’:

```
WITH RECURSIVE rec_prereq AS (  
  SELECT course_id, prereq_id  
    FROM prereq  
   WHERE course_id = 'DSC-100'  
  UNION  
  SELECT r.course_id, p.prereq_id
```

```

        FROM rec_prereq r
        JOIN prereq p
          ON r.prereq_id = p.course_id
    )
SELECT * FROM rec_prereq;

```

Base case finds direct prerequisites; recursive case adds one more level each iteration.

Example 3.2 (Reachable Cities). On table `Flights(origin, destination)`, find all cities reachable from San Diego:

```

WITH RECURSIVE ReachableCities(city) AS (
    SELECT destination
      FROM Flights
     WHERE origin = 'San_Diego'
    UNION ALL
    SELECT f.destination
      FROM Flights f
     JOIN ReachableCities rc
       ON f.origin = rc.city
)
SELECT city FROM ReachableCities;

```

This collects each new destination until no more appear.

Example 3.3 (Employee Hierarchy). Given `employees(employee_id, employee_name, manager_id)`, build the full reporting tree:

```

WITH RECURSIVE employee_hierarchy AS (
    -- Top-level managers:
    SELECT employee_id, employee_name, manager_id
      FROM employees
     WHERE manager_id IS NULL
    UNION ALL
    -- Add direct reports iteratively:
    SELECT e.employee_id, e.employee_name, e.manager_id
      FROM employees e
     JOIN employee_hierarchy eh
       ON e.manager_id = eh.employee_id
)
SELECT * FROM employee_hierarchy;

```

Starts from root managers and pulls in all descendants.

4 Pitfalls and Problematic Examples

Example 4.1 (Non-terminating Recursion). This view never reaches a fix-point unless externally limited:

```
CREATE RECURSIVE VIEW t AS
  SELECT 0 AS N
    FROM r
  UNION
  SELECT N+1 AS N
    FROM t;
```

Each iteration increments N forever.

5 Query Evaluation

Definition 5.1 (Query Evaluation). Query evaluation is the process of executing a query to retrieve data from a database.

Query evaluation proceeds in three main phases:

1. **Parsing:** Analyze the query's syntax and convert it into an internal representation (e.g. a parse tree).
2. **Optimization:** Generate a query plan that minimizes resource usage and execution time, selecting both logical (relational-algebra) and physical operators. 22
3. **Execution:** Retrieve data from disk or memory and process it according to the chosen plan.

5.1 Relational Algebra and Physical Operators

Logical Operators $\cup, \cap, -, \sigma, \pi, \times, \bowtie, \rho, \delta, \gamma, \tau$.

Physical Algorithms Each logical operator can be implemented by multiple algorithms, for example:

- Main-memory hash join
- External-memory merge join
- Partitioned hash join
- Sort-based grouping

Example 5.1 (Main-Memory Join Algorithms). For the relational-algebra expression

$$\text{Movie} \bowtie_{\text{Movie.mid}=\text{ActedIN.mid}} \text{ActedIN},$$

when both tables fit in main memory, possible implementations are:

1. **Nested-loop join:** $O(n^2)$.
2. **Merge join:** $O(n \log n)$ if inputs are pre-sorted.
3. **Hash join:** $O(n)$ on average, worst-case $O(n^2)$.

5.2 Hash Table Review

Definition 5.2 (Hash Table). A hash table stores key–value pairs using a hash function $h(k)$ to map keys into buckets; collisions are handled (e.g. by separate chaining).

Definition 5.3 (Insert and Find). **insert**(k, v) adds value v under key k . Duplicate keys are allowed. **find**(k) returns the list of all values associated with key k .

5.3 Combining Operators

Definition 5.4 (Materialization). Materialized evaluation computes an operator’s entire output, stores it in a temporary relation, then feeds it to the next operator. Pros: allows reuse; cons: additional I/O and storage.

Definition 5.5 (Pipelining). Pipelined evaluation streams tuples directly from one operator to the next without fully materializing intermediate results, reducing I/O and memory usage but limiting reuse.

Database systems use hybrids of materialization and pipelining based on data sizes, operator costs, and resource availability.

5.4 Physical Plan Decisions

A physical plan must specify:

- The algorithm for each operator (e.g. nested-loop vs. hash join).
- Whether to pipeline or materialize between operators.
- The access path for each base relation (full scan vs. index scan).

6 Data Storage and Indexing

6.1 Memory Hierarchy

Definition 6.1 (Registers). CPU registers are the fastest storage (0.25–0.5 ns), smallest capacity (bytes–KB), volatile, and most expensive per bit.

Definition 6.2 (Cache Memory). Cache (SRAM) is organized in L1/L2/L3 levels (0.5–25 ns), volatile, stores frequently accessed data to reduce main-memory accesses.

Definition 6.3 (Main Memory). DRAM (4 GB–32 GB) with access times 80–250 ns, volatile, balances cost and capacity.

Definition 6.4 (Secondary Storage). HDDs/SSDs (hundreds of GB–TB), persistent, slowest access (μ s–ms), most cost-effective per bit.

6.2 Disk Organization

Definition 6.5 (Hard Disk Structure). A disk consists of platters, tracks (concentric circles), sectors (smallest addressable units), a read/write head on an actuator arm, and a spindle motor.

Definition 6.6 (Rotational Latency). Time for the disk to rotate the desired sector under the head; inversely proportional to RPM.

Data is transferred in blocks (pages); sequential block reads are far more efficient than random reads.

6.3 Data Files

Definition 6.7 (Heap File). An unsorted file of records; insertion is fast but queries require scanning.

Definition 6.8 (Sequential File). Records sorted by a key, enabling binary search and efficient range queries.

6.4 Indexes

Definition 6.9 (Index). An auxiliary file mapping search keys to record pointers. Entries are (key, pointer).

Index Types

- **Hash Index:** Fast for point queries.
- **B+-Tree Index:** Supports both point and range queries.
- **Clustered vs. Unclustered:** Clustered index dictates physical order (one per table); unclustered does not.
- **Primary vs. Secondary:** Primary covers the primary key, secondary covers other attributes.

Example 6.1 (Creating Indexes). `CREATE INDEX idx_V_N ON V(N);`
`CREATE INDEX idx_V_P_M ON V(P, M);`

Index `idx_V_N` speeds up `WHERE N = ?`; composite index `idx_V_P_M` speeds up `WHERE P = ? AND M = ?`.

Example 6.2 (Index Nested-Loop Join). `SELECT *`
`FROM Student x, Takes y`
`WHERE x.ID = y.studentID`
`AND y.courseID > 300;`

With indexes on `Takes(courseID)` and `Student(ID)`, the engine:

1. Uses `Takes(courseID)` to fetch matching *y*.
2. For each *y*, uses `Student(ID)` to fetch *x*.

This is the *index nested-loop join*.

6.5 Index Selection Guidelines

- Index attributes appearing in `WHERE` (equality or range) or join conditions.
- Use composite indexes when multiple columns frequently appear together.
- Cluster indexes for range-heavy access (1–2% of tuples).
- Weigh index maintenance overhead against query performance gains.

7 Design Theory

7.1 Database Design Process

The database design process proceeds through several levels of modeling:

- **Conceptual Model:** High-level entities and relationships (e.g. `Company` makes `Product`).
- **Relational Model:** Tables with attributes and integrity constraints.
- **Normalization:** Decomposition to eliminate anomalies (redundancy, update, deletion).
- **Physical Schema:** File organization and storage structures.

7.2 Anomalies in Unnormalized Schemas

Example 7.1 (Person Phone–City Schema). Consider a single relation:

`Person`(`Name`, `SSN`, `PhoneNumber`, `City`)

with instance:

Name	SSN	PhoneNumber	City
Fred	123-45-6789	619-555-1234	San Diego
Fred	123-45-6789	619-555-6543	San Diego
Joe	987-65-4321	213-555-2121	LA

- *Redundancy:* `City` repeated for Fred’s two phones.
- *Update anomaly:* Moving Fred to “LA” requires updating multiple tuples.
- *Deletion anomaly:* Deleting Joe’s only phone would lose his city information.

7.3 Relation Decomposition

To remove anomalies, decompose `Person` into:

`PersonInfo`(`Name`, `SSN`, `City`), `PersonPhone`(`SSN`, `PhoneNumber`)

This eliminates redundancy and all update/deletion anomalies.

7.4 Functional Dependencies (FDs)

Definition 7.1 (Functional Dependency). Given a relation schema R , an FD

$$A_1, A_2, \dots, A_n \rightarrow B_1, B_2, \dots, B_m$$

holds on R if for every instance r of R and all tuples $t, t' \in r$,

$$(t[A_1] = t'[A_1] \wedge \dots \wedge t[A_n] = t'[A_n]) \implies (t[B_1] = t'[B_1] \wedge \dots \wedge t[B_m] = t'[B_m]).$$

Example 7.2 (EmpID and Position FDs). On $\text{Emp}(\text{EmpID}, \text{Name}, \text{Phone}, \text{Position})$:

- $\text{EmpID} \rightarrow \{\text{Name}, \text{Phone}, \text{Position}\}$.
- $\text{Position} \rightarrow \text{Phone}$.
- $\text{Phone} \not\rightarrow \text{Position}$.

7.5 Inference of FDs: Closure

Definition 7.2 (Attribute Closure). For a set of attributes X in schema R with FD set $\mathcal{F}$, the *closure* X^+ is all attributes functionally determined by X . Compute X^+ by:

$$\begin{aligned} X^+ &\leftarrow X; \\ \text{repeat while } X^+ \text{ grows:} \\ \text{for each } Y \rightarrow Z \in \mathcal{F} \text{ s.t. } Y \subseteq X^+, X^+ &\leftarrow X^+ \cup Z. \end{aligned}$$

Example 7.3 (Computing Closure). Let $R(A, B, C, D, E, F)$ with FDs:

$$A, B \rightarrow C, \quad A, D \rightarrow E, \quad B \rightarrow D, \quad A, F \rightarrow B.$$

- $\{A, B\}^+ = \{A, B, C, D, E\}$.
- $\{A, F\}^+ = \{A, F, B, C, D, E\}$.

7.6 Keys and Superkeys

Definition 7.3 (Superkey and Key). A *superkey* is $X \subseteq R$ such that X^+ contains all attributes of R . A *key* is a minimal superkey (no proper subset is a superkey).

Example 7.4 (Key for Product). For $\text{Product}(\text{Name}, \text{Price}, \text{Category}, \text{Color})$ with FDs $\text{Name}, \text{Category} \rightarrow \text{Price}$ and $\text{Category} \rightarrow \text{Color}$:

$$(\text{Name}, \text{Category})^+ = \{\text{Name}, \text{Category}, \text{Price}, \text{Color}\},$$

so $(\text{Name}, \text{Category})$ is a key.

7.7 Eliminating Anomalies via Normalization

An FD $X \rightarrow A$ is *safe* only if X is a superkey; otherwise decompose the relation on FDs to achieve a normal form free of anomalies.

8 Entity–Relationship (E/R) Diagrams

8.1 Motivation

Suppose a healthcare company needs a data analytics platform to track patient outcomes across various medical procedures. As a data analyst, you must:

- Elicit data requirements by interviewing stakeholders and examining sample records.
- Create a conceptual data model to capture entities, attributes, relationships, and constraints.
- Translate that conceptual model into a physical schema (tables, columns, indexes, constraints).

8.2 Basic Concepts

Definition 8.1 (Entity Set). An *entity set* is a collection of similar real-world objects, e.g. **Patient**, **Procedure**, or **Doctor**.

Definition 8.2 (Entity). An *entity* is an individual object in an entity set, such as a particular patient or a specific medical procedure.

Definition 8.3 (Attribute). An *attribute* is a property of an entity or relationship, for example, a patient’s **PatientID**, **Name**, or **DateOfBirth**.

Definition 8.4 (Relationship). A *relationship* is an association among two or more entity sets, such as “performs” between **Doctor** and **Procedure**.

8.3 Keys in E/R Diagrams

Definition 8.5 (Key). A *key* for an entity set is an attribute (or minimal combination of attributes) that uniquely identifies each entity in that set, e.g. **PatientID** for **Patient**.

8.4 Mathematical View of Relationships

Definition 8.6 (Relation). If A and B are sets, a *relation* $R \subseteq A \times B$ is any subset of ordered pairs. For example,

$$A = \{1, 2, 3\}, \quad B = \{a, b, c, d\}, \quad A \times B = \{(1, a), (1, b), \dots, (3, d)\},$$

and one relation might be $R = \{(1, a), (1, c), (3, b)\}$.

Example: A `makes` relationship between `Product` and `Company` is a subset of `Product` $\times$ `Company`.

8.5 Cardinality Constraints

- **One–One:** Each entity in set A relates to at most one in B , and vice versa.
- **One–Many:** Each entity in A relates to many in B , but each in B relates to at most one in A .
- **Many–Many:** Entities in A and B may relate to multiple counterparts.

8.6 Attributes on Relationships

Relationships can carry attributes of their own. **Example:** A `Buys` relationship between `Person` and `Product` might include an attribute `PurchaseDate`.

8.7 Multi-way (Higher-arity) Relationships

A relationship may involve three or more entity sets. **Example:** A ternary `Purchase` among `Person`, `Product`, and `Store` is modeled as a set of triples

$$\subseteq \text{Person} \times \text{Product} \times \text{Store}.$$

Cardinality arrows indicate constraints across any two dimensions.

8.8 Converting Multi-way to Binary Relationships

To implement in a relational schema, decompose a ternary relationship into three binary relationships (or a single table with foreign keys to all three entity tables), ensuring cardinality constraints are preserved.

8.9 Design Principles

- **Stay faithful to requirements:** Model exactly what the application specifies—no more, no less.
- **Avoid unnecessary complexity:** Don't introduce extra entities or relationships if simpler constructs suffice.
- **Use appropriate entity types:** Distinguish when something is truly an entity versus merely an attribute or relationship.

8.10 Mapping E/R Diagrams to Relational Schemas

- *Entity set* $\rightarrow$ a table with columns for each attribute and a primary key.
- *Relationship*
 - One–One or One–Many: add a foreign key in the “many” side.
 - Many–Many: create a junction table with foreign keys to each participating entity.
 - Multi-way: create a table with foreign keys to all involved entities plus relationship attributes.

8.11 Modeling Subclasses and Union Types

- Use ISA to define subclasses (specialized entity sets).
- Translate via *single-table*, *class-table*, or *concrete-table* inheritance strategies.
- For union types (an entity may belong to one of several superclasses), introduce a shared “owner” entity or use alternative designs to enforce mutual exclusion.

8.12 Weak Entity Sets

A *weak entity set* has no key of its own; its key is formed by combining a discriminator with the primary key of its owning (identifying) entity set. **Example:** A **Team** weakly owned by **University**, identified by {**UniversityID**, **TeamNumber**}.

9 Boyce–Codd Normal Form (BCNF)

9.1 Summary

- **Anomalies:** Redundancy in table records can lead to insertion, deletion, and update anomalies.
- **Decomposition:** We remove anomalies by decomposing a table into smaller tables that preserve the original data.
- **Functional Dependencies (FDs):** FDs capture how attributes determine one another and guide decomposition.
- **BCNF:** A strong normal form that ensures no “bad” FDs remain, thereby eliminating anomalies.

9.2 Definition of BCNF

A relation schema R is in *Boyce–Codd Normal Form* if for every non-trivial functional dependency

$$X \rightarrow Y$$

holding on R , the set of attributes X is a superkey of R . Here “non-trivial” means $Y \not\subseteq X$.

Equivalently, for every FD $X \rightarrow Y$ in R either

- the FD is trivial ($Y \subseteq X$), or
- the closure X^+ (under all FDs of R) equals the full set of attributes of R .

9.3 BCNF Decomposition Algorithm

Normalize(R):

```
find a set of attributes  $X \subset R$  such that:  
   $X^+ = R$  and  $X \neq R$  (all attributes of  $R$ )  
if no such  $X$  exists:  
  report " $R$  is in BCNF"  
else:  
  let  $Y = R \setminus X$   
   $Z = R \setminus X^+$   
  decompose  $R$  into  
     $R_1 = X \cup Y$  and  $R_2 = X \cup Z$ 
```


Normalize(R1)

Normalize(R2)

9.4 Example: A “Bad” FD

Consider a relation

PersonPhoneCity(Name, SSN, PhoneNumber, City)

with the single key {SSN, PhoneNumber}. The FD

$$\text{SSN} \rightarrow \{\text{Name}, \text{City}\}$$

is non-trivial and {SSN} is not a superkey, so this FD violates BCNF.

9.5 BCNF Decomposition: Person–Phone–City

1. **Decompose** PersonPhoneCity on the FD $\text{SSN} \rightarrow \{\text{Name}, \text{City}\}$ into
PersonInfo(SSN, Name, City) and Phone(SSN, PhoneNumber).
2. Both tables are now in BCNF:
 - In PersonInfo, SSN is a key.
 - In Phone, {SSN, PhoneNumber} is the key.
3. **Anomalies removed:** No redundancy of City or Name, and no update/deletion anomalies remain.

9.6 Example: Two-Step BCNF Decomposition

Relation

Person(Name, SSN, age, hairColor, phoneNumber)

with FDs

$$\text{SSN} \rightarrow \{\text{Name}, \text{age}\}, \quad \text{age} \rightarrow \text{hairColor}.$$

1. Compute $\{\text{SSN}\}^+ = \{\text{SSN}, \text{Name}, \text{age}, \text{hairColor}\}$. Decompose into
 $P_1(\text{SSN}, \text{Name}, \text{age}, \text{hairColor}), \quad P_2(\text{SSN}, \text{phoneNumber}).$
2. On P_1 , compute $\{\text{age}\}^+ = \{\text{age}, \text{hairColor}\}$. Decompose into
 $P_{11}(\text{SSN}, \text{Name}, \text{age}), \quad P_{12}(\text{age}, \text{hairColor}), \quad P_2(\text{SSN}, \text{phoneNumber}).$
3. All resulting schemas are in BCNF.

9.7 Example: $R(A, B, C, D)$ with $A \rightarrow B, B \rightarrow C$

- Key of R is $\{A, D\}$ since $A^+ = \{A, B, C\} \neq \{A, B, C, D\}$.
- Decompose on $A \rightarrow B$:

$$R_1(A, B, C), \quad R_2(A, D).$$

- Both R_1 and R_2 are in BCNF:

$$R_1(A, B, C) : A \rightarrow B, B \rightarrow C \Rightarrow A \text{ is a key.}$$

$$R_2(A, D) : \text{no non-trivial FDs, so trivially BCNF.}$$

9.8 Lossless vs. Lossy Decomposition

A decomposition of R into S_1 and S_2 is *lossless* if

$$R = S_1 \bowtie S_2.$$

Example 9.1 (Lossy Decomposition). Original $R(\text{Name}, \text{Price}, \text{Category})$:

$\{(\text{Gizmo}, 19.99, \text{Gadget}), (\text{OneClick}, 24.99, \text{Camera}), (\text{Gizmo}, 19.99, \text{Camera})\}$.

Decompose into

$$S_1(\text{Name}, \text{Category}), \quad S_2(\text{Price}, \text{Category}),$$

the join $S_1 \bowtie S_2$ yields an extra tuple $(\text{OneClick}, 19.99, \text{Gadget})$, so the decomposition is *lossy*.

9.9 Guaranteed Lossless Decomposition

If $X \rightarrow Y$ holds on $R(A_1, \dots, A_n, Y_1, \dots, Y_m, Z_1, \dots, Z_p)$, then

$$\Pi_{A_1, \dots, A_n, Y_1, \dots, Y_m}(R) \text{ and } \Pi_{A_1, \dots, A_n, Z_1, \dots, Z_p}(R)$$

form a lossless decomposition.

9.10 Testing Lossless Join: The Chase Test

To test $R = S_1 \bowtie \cdots \bowtie S_k$, one constructs a *chase tableau* and repeatedly applies the FDs to equate symbols. If at any point a row becomes identical to the “identity” row, the join is lossless.

Example 9.2 (Chase for $R(A, B, C, D)$ with $A \rightarrow B$, $B \rightarrow C$, $CD \rightarrow A$).
Decompose into

$$S_1(A, D), \quad S_2(A, C), \quad S_3(B, C, D).$$

A few chase steps suffice to show all attributes collapse to consistent symbols, proving the decomposition is lossless.

9.11 Schema Refinements and Normal Forms

- **1NF:** All tables are flat (atomic attributes).
- **2NF:** Obsolete in modern practice.
- **BCNF:** No non-trivial FD has a non-superkey determinant.
- **3NF:** Allows some non-key FDs but preserves all FDs while avoiding most anomalies.

BCNF guarantees no redundancy anomalies but may “lose” certain FDs; 3NF preserves all FDs at the expense of allowing some benign anomalies.